

# PA3: Planning

Xiaoming Zhao

Screen Recording: [Google Drive Link]

May 4, 2026

## Implementation

A ROS 2 node `pa3_path_planner` with `nav_msgs/OccupancyGrid` on `/map`, plan path with BFS, DFS, or A\*, and drives the rosbot to goal.

The robot's pose in the `map` frame is obtained via `tf2_ros.Buffer.lookup_transform("map", "base_link")`; the goal is read from stdin; the algorithm and connectivity are selected at launch with ROS parameters.

A discrete Gaussian kernel with  $5 \times 5$ ,  $\sigma=1.3$  is implemented with the raw occupancy. Unknown cells ( $-1$ ) are excluded from the average so inflation does not bleed into unexplored space. The smoothed grid is published on `/map_smoothed`; also, a hard obstacle mask is built and dilated by  $R=4$  cells to ensure planned path keeps clear of walls. BFS and DFS use a visited set so they cannot cycle. A\* uses `heapq` as priority queue with an admissible heuristic. Manhattan for 4-connectivity, octile  $h(n)=\Delta r+\Delta c+(\sqrt{2}-2)\min(\Delta r, \Delta c)$  for 8-connectivity. Edge cost is 1 for cardinals,  $\sqrt{2}$  for diagonals; in weighted mode the cost is multiplied by  $1+b_{rc}$  where  $b_{rc} \in [0, 1]$  is the normalized blurred occupancy.

The cell sequence is down sampled by `path_stride`, mapped to world coordinates with the grid origin and resolution, and used as `geometry_msgs/PoseArray` on `/pose_sequence`. Each pose's yaw points to the next waypoint,  $\theta_i=\text{atan2}(y_{i+1}-y_i, x_{i+1}-x_i)$ , so the robot faces  $i+1$  before moving. For the motion control, proportional waypoint follower at 20 Hz reads the tf and the waypoint. When yaw error  $|\psi_e| > 0.4$  rad it rotates in place; otherwise  $v=\min(v_{\max}, k_\ell d \cos \psi_e)$  and  $\omega=\text{sat}_{w_{\max}}(k_a \psi_e)$ , with  $k_\ell=0.5$ ,  $k_a=1.5$ ,  $v_{\max}=0.25$  m/s,  $w_{\max}=1.2$  rad/s. Higher gains caused wall scraping and oscillation in narrow corridors, and waypoint advance triggers at 0.15 m, goal completion at 0.10 m.

## Evaluation

All runs use the  $200 \times 200$  `maze.world`, 8-connectivity,  $R=4$  inflation. Same start (1, 9) and goal (5, 5).

| Algorithm | Cells | Length (m) | Plan time |
|-----------|-------|------------|-----------|
| A*        | 98    | 5.66       | 2 ms      |
| BFS       | 185   | 8.10       | 3 ms      |
| DFS       | 975   | 41.2       | 92 ms     |

DFS finds a path but not an optimal one. As the 975 cell trajectory traversed every corridor in the maze before reaching the goal. BFS minimizes step count but is blind to diagonal length ( $\sqrt{2}$  vs. 1) and to the soft cost field. A\* with heuristic and weighted edges produced the shortest and smoothest trajectory as expected.

## Raw vs. Smoothed Path Comparison

The raw map without Gaussian and blur dilation, 8-conn weighted A\* returns the shortest path, but it collides with walls, since the minimum clearance to obstacles is one cell 0.05 m, inside the rosbot's 0.20 m half-width. Adding the  $5 \times 5$  Gaussian raises the cost near walls and pulls the path inward, but with low `inflate_thresh` alone the clearance is still inconsistent  $\sim 0.10$  m. Combining the blur with  $R=4$  hard dilation produces a path whose minimum clearance is 0.20 m and interior cells follow a corridor centre-line; total length grows from 5.42 m (raw) to 5.66 m. The trade is about using longer paths to exchange for no collision.

## Credits

Developed by Xiaoming Zhao, referencing lecture materials, ROS 2 `tf2/nav_msgs` documentation, and the `stage_ros2` source. Debugging, parameter tuning, and report formatting assisted by Generative AI, including Claude and Gemini.